Factored

WORKSHOP CHEATSHEET

Knowledge Graphs & Ontologies

1. Core Concepts

Knowledge Graphs organize massive, complex data into flexible networks of nodes and edges, transforming raw facts into meaningful contextual relationships. Unlike tabular data representations (like spreadsheets or relational databases) where connections are hard to express, a knowledge graph explicitly highlights interactions, making it easier to discover hidden insights and patterns.



Ontology vs. Knowledge Graph

An **Ontology** is the foundation, blueprint, or rulebook of a knowledge graph. It describes the domain (e.g., a Supermarket). Think of an ontology as the city planning documents (declaring where buildings go and what a street is), while the knowledge graph is the physical city itself—the actual populated data.

RDF & OWL Languages

RDF (Resource Description Framework) is the language used to represent complex relationships, organizing information into triples: *Subject* → *Predicate* → *Object*.
OWL (Web Ontology Language) adds richer vocabulary and rules to describe those subjects, allowing us to define classes, subclasses, and logical constraints.

Why build an Enterprise Knowledge Graph?

Summarizing all of a company's information in a system prompt for an AI agent is impossible due to token limits. An Ontology/KG compresses company structure and processes into a semantic layer an agent can easily navigate. Key benefits include:

- **Scalable Semantic Structure:** Organizes data effortlessly as it grows.
- **Auditable & Version Controlled:** Changes to the data model are traceable.
- **Unified Semantic Layer:** Multiple teams use the same consistent rules and knowledge.
- **Agentic Orchestration:** Powers intelligent agents and enterprise search.

2. OWL/RDF Syntax Guide

The following commands are used in our provided `.ttl` (Turtle) practice ontology file to define the supermarket domain.

Command	Educative Explanation
`@prefix`	Creates shorthand abbreviations for long web URLs, so we can write `:Item` instead of typing out a full web address every time.
`a`	A specialized RDF shorthand for "is a" (`rdf:type`). It declares what category an entity belongs to.
`owl:Class`	Defines a broad category or template in our domain, such as a `:Store` or an `:Item`.
`owl:ObjectProperty`	Defines a relationship between two entities (Node to Node). Example: `:locatedInDivision` connects a Store to a Division.
`owl:DatatypeProperty`	Defines an attribute of an entity, connecting a Node to a literal value like text or a number. Example: `:storeId` connects a Store to a String.
`rdfs:domain`	Specifies the starting point (the Subject) of a property. If the domain is `:Store`, only stores can have this property.
`rdfs:range`	Specifies the valid target (the Object or Data type) of a property. For `:storeId`, the range is `xsd:string`.
`rdfs:subClassOf`	Creates a hierarchy by stating one class is a specific type of another. Example: A `:PercentOffPromo` is a sub-class of a `:Promo`.

3. Project Files

Knowledge graph and ontology

- ontology/ontology.owl.ttl The schema: defines the supermarket's classes (such as Store and Item), relationships, and attributes. Load it in a graph viewer to explore how the domain is modelled.
- ontology/your_own_ontology.owl.ttl A mostly empty ontology template for participants to define their own classes and properties.

These Turtle (.ttl) files are data, not programs to run directly. Open them with a graph viewer or use them as input to the Python assistant.

Graph viewers

- graph-view/live-editor.html An interactive Turtle editor that visualizes the graph as you type; it can also open and save .ttl files. Open it in a browser. It loads CodeMirror from a CDN, so that editor component needs internet access.
- graph-view/vis-network.min.js The bundled graph-drawing library used by the viewers. It is a dependency, not a file you normally run or edit.

To start either viewer, double-click its .html file or open it from your browser. The visualizer uses the bundled library; the live editor also needs internet access for CodeMirror.

Assistant and web app

- src/ontology_agent.py Loads the ontology and sample data, asks an LLM to answer questions, and runs its generated SPARQL queries against the local graph.
- src/server.py Starts the Flask server and provides the ontology and question-answer API.
- src/ui/index.html The browser interface for chatting with the assistant and viewing graph/query-traversal information. It is served by Flask; don't open it directly as a file.